

Off-Policy GRPO for Online Speculative Decoding

MSAI-635 Project Proposal

Gaurav Prakash
UID: 122250130

Sanjana Kommareddy
UID: 122099364

Simran Kharbanda
UID: 122283671

1 Problem Significance

Background. Speculative decoding accelerates LLM inference by letting a small *draft* model propose a block of K tokens that the large *target* model verifies in a single forward pass. Under the standard speculative-sampling rule, the i -th drafted token is accepted with probability $\min(1, (a_i)/(a_i))$ and verification stops at the first rejection, so the output distribution is *exactly* that of the target. The speed-up is governed almost entirely by the *acceptance length*: the number of drafted tokens the target accepts per verification step.

Why speculative decoding must be an online problem. The base paper we are using, Aurora [1], argues that this offline “train-then-deploy” pipeline is fundamentally mismatched to production serving:

- **Distribution shift.** Acceptance depends on the *live* traffic distribution (domain, language, task), which drifts over time; a drafter trained on yesterday’s corpus decays on today’s traffic.
- **Day-0 deployment.** Newly released frontier targets have no trained drafter. Waiting for an offline data-generation and training cycle forfeits speed-up exactly when demand is highest.

Aurora couples an SGLang *inference server* with a *training server*: serving traces (target hidden states and logits) are streamed through a Mooncake store to FSDP training workers, and updated draft weights are hot-swapped back into the server. Aurora frames this loop as asynchronous reinforcement learning (the server is the environment, the drafter is the policy), yet the loss it actually optimizes is a forward-KL *distillation* objective. We keep Aurora’s system and replace that loss.

Significance. Distillation is only a *proxy* for what we deploy for. The expected per-token acceptance is $\alpha = \sum_v \min((v), (v)) = 1 - \text{TV}(\cdot, \cdot)$, and the expected tokens per step is $1 + \sum_{k=1}^K \prod_{i \leq k} \alpha_i$, which compounds along the block. Forward KL spends capacity matching the target’s low-probability tail, weights all positions equally, and ignores that a mistake at position 1 nullifies every later position. Optimizing acceptance *directly* with a reward, using rewards the inference server already observes, aligns the training objective with the serving metric and makes Aurora’s RL framing literal rather than conceptual.

2 Problem Formulation

Speculation as an episodic MDP. Each verification step is one episode. The **state** $s_i = (x_{<t}, h_t, a_{<i})$ contains the committed context, the target’s auxiliary hidden states h_t , and the draft tokens proposed so far in the block. The **action** $a_i \in \mathcal{V}_{\text{draft}}$ is the next drafted token sampled from the **policy** $(\cdot | s_i)$. The **environment** is the target model’s verifier, which returns the accepted length $L(a) \in \{0, \dots, K\}$ of block $a = (a_1, \dots, a_K)$ (or of each root-to-leaf path in a draft tree). The **reward** is

$$R(a) = L(a) - \lambda \kappa (K - L(a)) + \eta 1[L(a) = K], \quad (1)$$

where κ is the relative cost of a wasted draft token (draft-to-target latency ratio), λ trades acceptance against wasted compute, and η is an optional bonus for full-block acceptance. With $\lambda = \eta = 0$, R is exactly the quantity that determines speed-up.

Asynchrony and off-policy. Training and serving run concurrently. The server samples with a *behavior* policy $= q_{\theta_{v-\Delta}}$, a copy of the drafter that is Δ weight-versions stale, while the trainer updates the current . Traces therefore arrive off-policy, with a time-varying staleness Δ set by the weight-sync interval and queueing in the transfer store. The context distribution $\mathcal{D}_\tau$ is also non-stationary.

Objective. Using only stale-policy traces and without slowing the server, learn

$$\max_{\theta} E_{x \sim \mathcal{D}_\tau} E_{a \sim (\cdot|x)} [R(a)] \quad \text{subject to losslessness of the target output distribution.} \quad (2)$$

Losslessness is guaranteed by the verifier for *any* , so RL can be applied safely: a poor policy can only cost speed, never quality.

3 Individual Contributions

Member	Area	Responsibilities
Simran	Inference server & reward design	SGLang inference srver setup and patch for stochastic draft-tree sampling; reward function design (Eq. 1) and group-normalized advantage computation; server-side trace pipeline; versioned weight hot-swap.
Gaurav	Training server & RL algorithm	Replace the forward-KL loss with the off-policy GRPO objective; KL anchor and β annealing; FSDP2 integration and stability tuning.
Sanjana	Evaluation & experiments	baseline experiments (static EAGLE-3, Aurora-KL); ablations on group size, KL anchor and reward shaping; report and plots.
Simran & Gaurav	Server-trainer interface	Jointly define the trace schema, the shared reward/advantage module and the weight-version protocol that link the two servers.

All members jointly own the end-to-end integration test and final write-up.

4 Methodology

Groups of rollouts. GRPO needs G rollouts from the same state to form a baseline without a critic. With EAGLE tree drafting ($\text{topk} > 1$), the target verifies *all* root-to-leaf paths of the draft tree in one forward pass, so the G paths from one context form a natural group whose rewards come from the real verifier.

Off-policy GRPO objective. For a group $\{a^{(j)}\}_{j=1}^G$ from context s , we use the group-normalized advantage and per-token importance ratio

$$\hat{A}_j = \frac{R_j - \text{mean}(R)}{\text{std}(R) + \epsilon}, \quad \rho_{j,i} = \frac{(a_i^{(j)} | s_i^{(j)})}{(a_i^{(j)} | s_i^{(j)})},$$

and optimize

$$\mathcal{J}(\theta) = E \left[\frac{1}{G} \sum_{j=1}^G \frac{1}{|a^{(j)}|} \sum_i w_{j,i} \min \left(\rho_{j,i} \hat{A}_j, \text{clip}(\rho_{j,i}, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_j \right) \right] - \beta \text{KL}(\parallel), \quad (3)$$

where $w_{j,i} = \min(\rho_{j,i}, \bar{c})$ is a truncated importance weight that bounds variance from stale traces, and samples with staleness $\Delta > \Delta_{\max}$ are discarded. The final term reuses Aurora’s existing forward-KL loss as a *dense anchor*: it stabilizes early (day-0) training when rewards are near zero, and β is annealed as acceptance improves. Positions after the first rejection receive no gradient through the reward term, matching the verifier’s semantics.

End-to-end asynchronous workflow. The two servers run concurrently and never block on each other: the inference server serves with whatever draft version it holds, logs traces and polls for new weights every N steps, while the trainer pulls traces, drops stale ones, takes a GRPO step and publishes updated weights.

Framework and experimental setup. We keep Aurora’s experimental setup unchanged so results are directly comparable to the KL baseline:

- **Stack:** PyTorch with FSDP2 and FlexAttention, Ray for orchestration, an SGLang inference server with EAGLE-3 speculative decoding, the Mooncake store for trace transfer, and W&B for logging. No extra RL library is needed; GRPO is implemented directly in PyTorch.
- **Two-server layout:** training on GPUs $\{0,1\}$; external SGLang server on GPU $\{2\}$ (Qwen3-4B, for development) or $\{2-5\}$ (Qwen3-8B, main experiments), sharing a filesystem for weight sync

5 Validation and Evaluation Metrics

Primary metric: target acceptance.

- **Mean acceptance length** $\bar{\tau}$: accepted draft tokens per verification step on the live server (headline metric).
- **Per-position acceptance** α_i for $i = 1, \dots, K$, and the **acceptance rate** (accepted / drafted tokens).

Composite and systems metrics.

- **End-to-end speed-up** over target-only decoding, throughput (tokens/s), TPOT and TTFT.

Verification. (i) *Losslessness*: with greedy decoding, outputs must be token-identical to target-only decoding; with sampling, we check that output token distributions match within statistical tolerance. (ii) *Correctness of the RL signal*: logged accepted lengths are cross-checked against counts recomputed offline from stored target logits. (iii) *Baselines and ablations*: static EAGLE-3, Aurora with forward-KL only, synchronous on-policy GRPO ($\Delta=0$), and our off-policy GRPO, with ablations over group size G , KL anchor β , and reward shaping (λ, η) .

Success criterion. Match or exceed Aurora-KL in $\bar{\tau}$ and speed-up on Qwen3-8B, adapt faster after domain shift (higher AUC), and remain lossless on the same two-server hardware budget.

6 References

- [1] J. Wang, F. Bie, J. Li, Z. Zhou, Z. Shao, Y. Wang, Y. Liu, Q. Wu, A. May, S. Yanamandra, Y. Zhang, C. Zhang, T. Dao, P. Liang, B. Athiwaratkun, S. L. Song, C. Xu, and X. Wu. When RL meets adaptive speculative training: A unified training–serving system. *ICML*, 2026.